

Matrix Computations

Chapter 1: Matrix Multiplication

Based on *Matrix Computations, 4th Edition*

Lecturer: Huiwen Wu

Lecture Overview

- 1 Basic Algorithms and Notation
- 2 Structure and Efficiency
- 3 Block Matrices and Algorithms
- 4 Fast Matrix-Vector Products
- 5 Vectorization and Locality
- 6 Parallel Matrix Multiplication
- 7 Summary and References

- Matrix multiplication is fundamental to matrix computations.
- Several equivalent formulations exist:
 - Dot product formulation
 - Saxpy formulation
 - Outer product formulation
- Each has different performance characteristics due to memory access.

Matrix Notation

- $\mathbb{R}^{m \times n}$: vector space of $m \times n$ real matrices.
- $\mathbb{R}^n$: column vectors.
- Entry of matrix A : a_{ij} , $[A]_{ij}$, or $A(i, j)$.
- Transpose: $(A^T)_{ij} = a_{ji}$.
- Addition: $(A + B)_{ij} = a_{ij} + b_{ij}$.
- Scalar multiplication: $(\alpha A)_{ij} = \alpha a_{ij}$.

Matrix Multiplication

Definition

$C = AB$, with $A \in \mathbb{R}^{m \times p}$, $B \in \mathbb{R}^{p \times n}$:

$$c_{ij} = \sum_{k=1}^p a_{ik} b_{kj}.$$

Pointwise (Hadamard) operations also exist:

- $C = A \bullet B \Rightarrow c_{ij} = a_{ij} b_{ij}$
- $C = A \oslash B \Rightarrow c_{ij} = a_{ij} / b_{ij}$ (if $b_{ij} \neq 0$)

E01: Dimensions, entries, and Hadamard products

$$A = \begin{bmatrix} 1 & 2 & 0 \\ -1 & 3 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & 1 \\ 0 & -1 \\ 4 & 2 \end{bmatrix}.$$

A is 2×3 , B is 3×2 , so AB is 2×2 .

$$c_{12} = 1(1) + 2(-1) + 0(2) = -1, \quad AB = \begin{bmatrix} 2 & -1 \\ 2 & -2 \end{bmatrix}.$$

Try it: Is $A \bullet B$ defined? What is the size of BA ?

Check

The Hadamard product is undefined here because the shapes differ. BA is 3×3 .

E01 figure: A row meets a column

A				B			$C = AB$	
1	2	0		2	1		2	-1
-1	3	1	$\times$	0	-1	$=$	2	-2
				4	2			

Highlighted row and column meet at c_{12}

$$[1, 2, 0] [1, -1, 2]^T = 1 - 2 + 0 = -1$$

Try it: Which row and column would you highlight to obtain c_{21} ?

Vector Operations

- Scalar-vector: $z = ax \Rightarrow z_i = ax_i$.
- Addition: $z = x + y \Rightarrow z_i = x_i + y_i$.
- Inner product (dot): $x^T y = \sum_i x_i y_i$.
- Saxpy: $y = ax + y$ (scalar a times x plus y).

Algorithm 1.1.1: Dot product

```
c = 0;  
for i = 1:n  
    c = c + x(i)*y(i);  
end
```

$O(n)$ flops.

E02: Trace a dot product

$x = (1, -2, 3)^T$, $y = (4, 0, -1)^T$; initialize $c = 0$.

i	x_i	$x_i y_i$	New accumulator c
1	1	4	4
2	-2	0	4
3	3	-3	1

$$x^T y = 4 + 0 - 3 = 1.$$

Try it: How many multiplications and additions does the displayed loop execute? Three multiplications and three additions, counting the first addition to zero.

An algebraic sum of three products needs only two additions.

Saxpy Algorithm

Algorithm 1.1.2: AXPY update

Overwrites y with $y + ax$:

```
for i = 1:n  
    y(i) = y(i) + a*x(i);  
end
```

n multiplications and n additions: $2n$ flops.

AXPY means ax plus y . In BLAS, SAXPY is the single-precision version; DAXPY uses double precision.

E03: An update and a gradient step

Let $a = 2$, $x = (1, -2, 3)^T$, and $y_{\text{old}} = (4, 0, -1)^T$.

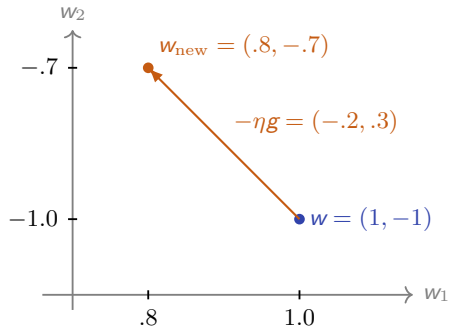
$$y_{\text{new}} = y_{\text{old}} + 2x = \begin{bmatrix} 4 + 2 \\ 0 - 4 \\ -1 + 6 \end{bmatrix} = \begin{bmatrix} 6 \\ -4 \\ 5 \end{bmatrix}.$$

Machine-learning connection: gradient descent is an AXPY update:

$$w_{\text{new}} = w - \eta g, \quad w = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad g = \begin{bmatrix} 2 \\ -3 \end{bmatrix}, \quad \eta = 0.1 \quad \Longrightarrow \quad w_{\text{new}} = \begin{bmatrix} 0.8 \\ -0.7 \end{bmatrix}.$$

Try it: Which quantities correspond to a , x , and y ?

E03 figure: A gradient step in parameter space



$$g = (2, -3) \text{ and } \eta = 0.1$$

$$w_{\text{new}} = w - \eta g$$

An AXPY update moves the parameter vector in the negative gradient direction.

Axes show a local window around the two iterates.

Matrix-Vector Multiplication (Gaxpy)

Generalized saxpy: $y = y + Ax$

Row-oriented (Algorithm 1.1.3):

```
for i = 1:m
    for j = 1:n
        y(i) = y(i) + A(i,j)*x(j);
    end
end
```

$O(mn)$ work.

Column-oriented (Algorithm 1.1.4):

```
for j = 1:n
    for i = 1:m
        y(i) = y(i) + A(i,j)*x(j);
    end
end
```

E04: The same gaxpy by rows and columns

$$A = \begin{bmatrix} 1 & 2 \\ -1 & 3 \\ 2 & 0 \end{bmatrix}, \quad x = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \quad y_0 = \begin{bmatrix} 1 \\ 0 \\ -2 \end{bmatrix}.$$

Rows: $Ax = (2 - 2, -2 - 3, 4 + 0)^T = (0, -5, 4)^T.$

$$y = y_0 + Ax = (1, -5, 2)^T.$$

Columns:

$$y_0 + 2A(:, 1) = \begin{bmatrix} 3 \\ -2 \\ 2 \end{bmatrix}, \quad \begin{bmatrix} 3 \\ -2 \\ 2 \end{bmatrix} - A(:, 2) = \begin{bmatrix} 1 \\ -5 \\ 2 \end{bmatrix}.$$

Try it: What would change if the initial vector were zero?

Partitioning Matrices

Row partition:

$$A = \begin{bmatrix} r_1^T \\ \vdots \\ r_m^T \end{bmatrix}, \quad r_k \in \mathbb{R}^n.$$

Column partition:

$$A = [c_1 \quad \cdots \quad c_n], \quad c_k \in \mathbb{R}^m.$$

Example: $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}.$

Colon Notation

- $A(k, :)$: k -th row.
- $A(:, k)$: k -th column.

Rewritten gaxpy:

```
% row-oriented
for i = 1:m
    y(i) = y(i) + A(i,:)*x;
end
% column-oriented
for j = 1:n
    y = y + A(:,j)*x(j);
end
```

E05: Read a slice before multiplying

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}.$$

Expression	Value	Shape
$A(2, :)$	$[3 \ 4]$	1×2
$A(:, 2)$	$(2, 4, 6)^T$	3×1
$A(1 : 2, :)$	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$	2×2

Try it: For $x = (2, -1)^T$, compute $A(2, :)x$ and $A(:, 2)x(2)$. **Check:** 2 (a scalar), and $(-2, -4, -6)^T$ (a column vector).

Outer Product Update

$A = A + xy^T$, with $x \in \mathbb{R}^m$, $y \in \mathbb{R}^n$.

- Row-oriented: for i , $A(i, :) = A(i, :) + x(i)y^T$.
- Column-oriented: for j , $A(:, j) = A(:, j) + y(j)x$.

$O(mn)$ flops.

E06: Build a rank-one update

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad y = \begin{bmatrix} 3 \\ -1 \\ 2 \end{bmatrix}, \quad xy^T = \begin{bmatrix} 3 & -1 & 2 \\ 6 & -2 & 4 \end{bmatrix}.$$

For $A_0 = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$,

$$A_0 + xy^T = \begin{bmatrix} 4 & -1 & 3 \\ 6 & -1 & 4 \end{bmatrix}.$$

Try it: Why does xy^T have rank one? Must $A_0 + xy^T$ have rank one? Every column of xy^T is a multiple of x . The updated matrix need not have rank one.

Matrix-Matrix Multiplication: Three Views

Let $A \in \mathbb{R}^{m \times r}$, $B \in \mathbb{R}^{r \times n}$, and initialize $C = 0$.

- **Dot product:** $c_{ij} = A(i, :)B(:, j)$.
- **Row saxpy update:** for each i, k , $C(i, :) \leftarrow C(i, :) + a_{ik}B(k, :)$.
- **Outer product accumulation:** $C = \sum_{k=1}^r A(:, k)B(k, :)$.

The dot view completes an entry. The saxpy view updates a row.

The outer-product view updates the entire output using a rank-one matrix.

E07: One product, three computational traces

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}, \quad AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}.$$

Dot: $c_{12} = 1 \cdot 6 + 2 \cdot 8 = 22$. **Row saxpy:** $C(1, :) = [0, 0] + 1[5, 6] + 2[7, 8] = [19, 22]$. **Outer products:**

$$AB = \begin{bmatrix} 1 \\ 3 \end{bmatrix} [5, 6] + \begin{bmatrix} 2 \\ 4 \end{bmatrix} [7, 8] = \begin{bmatrix} 5 & 6 \\ 15 & 18 \end{bmatrix} + \begin{bmatrix} 14 & 16 \\ 28 & 32 \end{bmatrix}.$$

Try it: Which view completes one entry, one row, or a whole-matrix contribution?

E07 figure: The product as two rank-one layers

$$\begin{array}{|c|c|} \hline A_{:,1} B_{1,:} \\ \hline 5 & 6 \\ \hline 15 & 18 \\ \hline \end{array} + \begin{array}{|c|c|} \hline A_{:,2} B_{2,:} \\ \hline 14 & 16 \\ \hline 28 & 32 \\ \hline \end{array} = \begin{array}{|c|c|} \hline AB \\ \hline 19 & 22 \\ \hline 43 & 50 \\ \hline \end{array}$$

Each outer product fills the whole matrix.
Adding both rank-one contributions gives the final product.

Try it: Why can each layer have rank one while their sum has rank two?

Algorithms for Matrix Multiplication

Algorithm 1.1.5 (ijk): initialize $C = 0$ to compute AB .

```
for i = 1:m
  for j = 1:n
    for k = 1:r
      C(i,j) = C(i,j)+A(i,k)*B(k,j);
    end
  end
end
```

Six loop orderings: *ijk*, *jik*, *ikj*, *jki*, *kij*, *kji*. Each has different data access patterns (Table 1.1.1).

Loop Orderings Characteristics

Order	Inner operation	Inner-loop access
ijk, jik	dot accumulation	A by row, B by column
ikj, kij	row saxpy	B by row, C by row
jki, kji	column saxpy	A by column, C by column

kij and kji organize the full calculation as outer-product updates.

Which access is contiguous depends on row-major or column-major storage.

Flops and Big-Oh

Operation	Dimensions	Flops
Dot product	$x, y \in \mathbb{R}^n$	$2n$
Saxpy	$a \in \mathbb{R}, x, y \in \mathbb{R}^n$	$2n$
Gaxpy	$A \in \mathbb{R}^{m \times n}$	$2mn$
Outer product	$A \in \mathbb{R}^{m \times n}$	$2mn$
Matmul	$A \in \mathbb{R}^{m \times r}, B \in \mathbb{R}^{r \times n}$	$2mnr$

Big-Oh: dot $O(n)$, matrix-vector $O(n^2)$, matrix-matrix $O(n^3)$.

E08: Count work before estimating runtime

For $A \in \mathbb{R}^{3 \times 2}$ and $B \in \mathbb{R}^{2 \times 4}$:

$$\underbrace{3 \cdot 4}_{12 \text{ output entries}} \times \underbrace{2}_{\text{inner terms}} = 24 \text{ multiply-add updates} = 48 \text{ flops.}$$

A direct sum without the initial addition to zero costs $12(2 + 1) = 36$ flops. For square dense products, doubling n multiplies the leading work by $2^3 = 8$.

Try it: Does doubling matrix size always make runtime exactly eight times larger? No. Cache behavior, vectorization, parallelism, and overhead also change.

Levels and BLAS

- **Level 1:** $O(n)$ data and work (e.g., dot, saxpy).
- **Level 2:** $O(n^2)$ data and work (e.g., gaxpy, outer product).
- **Level 3:** $O(n^2)$ data, $O(n^3)$ work (e.g., matrix multiply).

BLAS = Basic Linear Algebra Subprograms. Level-3 BLAS are key for high performance.

Structured Matrices

- Band matrices: lower bandwidth p , upper bandwidth q .
- Triangular: upper/lower.
- Diagonal, symmetric, permutation.
- Exploit structure for reduced storage and flops.

Band Matrices

Definition

A has lower bandwidth p if $a_{ij} = 0$ for $i > j + p$; upper bandwidth q if $a_{ij} = 0$ for $j > i + q$.

Example (8×5 , $p = 1$, $q = 2$):

$$\begin{bmatrix} \times & \times & \times & 0 & 0 \\ \times & \times & \times & \times & 0 \\ 0 & \times & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ 0 & 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 & \times \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$